CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering

LABORATORY MANUAL

Program: B.Tech. (CSE / CE / IT / AIML)

Course: CEUA301 - Competitive Programming Essentials

Semester: V

Academic Year: 2026–27

Course Outcomes

CO1 – Apply mathematical and number-theoretic foundations to formulate efficient solutions for computational problems.

CO2 – Analyze and solve graph-theoretic problems by selecting appropriate graph algorithms based on problem structure and constraints.

CO3 – Design dynamic programming solutions by formulating optimal state-transition models for multi-stage optimization problems.

CO4 – Construct greedy algorithms and justify their correctness for optimization problems exhibiting the greedy-choice property.

CO5 – Develop optimized solutions using advanced data structures and algorithmic paradigms to solve high-complexity problems within time and space constraints.

Lab Overview

Lab #	Hrs	Theme / Concept Area (CO)	Title
1	2 Hrs	Simulation on Strings & Linked Lists (CO5)	Simulation on Strings & Linked Lists
2	2 Hrs	Bit Manipulation Techniques (CO1)	Bit Manipulation Techniques
3	2 Hrs	Number Theory Foundations (CO1)	Number Theory Foundations
4	2 Hrs	Sliding Window & Greedy Optimization (CO4)	Sliding Window & Greedy Optimization
5	2 Hrs	Sequence Alignment & Interval Greedy (CO3)	Sequence Alignment & Interval Greedy
6	4 Hrs	Backtracking & Combinatorial Search (CO5)	Backtracking & Combinatorial Search
7	4 Hrs	Dynamic Programming on Sequences (CO3)	Dynamic Programming on Sequences
8	4 Hrs	Monotonic Stack & Two-Pointer Patterns (CO5)	Monotonic Stack & Two-Pointer Patterns
9	4 Hrs	Graph Algorithms (CO2)	Graph Algorithms
10	4 Hrs	Advanced String Dynamic Programming (CO3)	Advanced String Dynamic Programming

Laboratory Policies and Evaluation Scheme

Each laboratory session is of 2 hours duration for single-session labs and 4 hours for double-session labs (see individual lab headers). Students are expected to solve the assigned Main Problems and answer the Key Questions during/after implementation; these are compulsory. The Supplementary Problem and the Post Laboratory Work (LeetCode practice) are provided for additional practice and are not compulsory.

Each lab is evaluated out of 10 marks using a weighted rubric consistent across all labs, distributed as follows:

Evaluation Criteria (General Pattern)	Typical Weight
Correctness of program output, including edge cases	40%
Conceptual explanation of logic, approach, and optimizations	25%
Alternative-strategy reasoning and trade-off discussion	20%

Evaluation Criteria (General Pattern)	Typical Weight
Code structure, naming, and documentation	15%

Exact wording of each criterion is tailored per lab and given in full under each lab's Rubric section.

Table of Contents

Lab 1: Simulation on Strings & Linked Lists

Simulation on Strings & Linked Lists | Duration: 2 Hrs

CO Mapping	CO5 – Develop optimized solutions using advanced data structures and algorithmic paradigms to solve high-complexity problems within time and space constraints.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	2 Hours
Total Hours of Problem Definition Implementation	1.5 Hours
Total Hours of Testing / Evaluation	0.5 Hours

Lab Aim

To implement direct simulation-based solutions for linked-list merging, circular-array transformation, and digit-by-digit string arithmetic.

Problem 1

Combining the Sorted Registries

Two departments each maintain a sorted linked list of employee ID records. For an upcoming audit, the two registries must be combined into a single sorted linked list by splicing together the existing nodes, preserving the sorted order throughout.

Input: Heads of two sorted linked lists, list1 and list2.

Output: Head of the merged sorted linked list.

Example 1: *Input: list1 = [1,2,4], list2 = [1,3,4]*
Output: [1,1,2,3,4,4]

Example 2: *Input: list1 = [], list2 = []*
Output: []

Problem 2

Defusing the Circular Cipher

A bomb-disposal technician receives a circular sequence of numbers and a key k from an informant. To decrypt the sequence, every number must be simultaneously replaced: if k is positive, each number is replaced with the sum of the next k numbers (wrapping around the circular sequence); if k is negative, with the sum of the previous |k| numbers; if k is zero, with 0. Given the circular sequence and key, produce the fully decrypted sequence.

Input: Circular array code of length n, integer key k.

Output: The decrypted array of length n.

Example 1: *Input: code = [5,7,1,4], k = 3*
Output: [12,10,16,13]

Example 2: *Input: code = [1,2,3,4], k = 0*
Output: [0,0,0,0]

Problem 3

The Ancient Merchant's Ledger

An accountant working in a country whose currency system predates modern computing must compute the product of two numbers recorded as long digit strings on ledger stones, far beyond what any built-in numeric type can hold without overflow. The ledger-keeper's guild rules forbid converting the numbers into built-in integer types — the multiplication must be performed digit by digit, exactly as with pen and paper, and the result recorded as a new digit string.

Input: Two strings num1, num2, each representing a non-negative integer.

Output: A string representing the product of num1 and num2.

Example 1: *Input: num1 = "2", num2 = "3"*
Output: "6"

Example 2: *Input: num1 = "123", num2 = "456"*
Output: "56088"

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze why merging two sorted linked lists by splicing existing nodes (rather than creating a new list) requires careful handling of a dummy-head pointer.
2. **[Problem 2]** ***(Evaluate)*** Evaluate the circular-array decryption approach: how does treating the array as circular (using modular indexing) simplify handling the wrap-around case compared to explicit boundary checks?
3. **[Problem 3]** ***(Create)*** Design the digit-by-digit multiplication approach for the Ancient Merchant's Ledger problem, explaining how partial products are accumulated into the correct positions of the result array without overflow.

Supplementary Problem

Splicing the Long-Digit Ledgers

The same merchant's guild also keeps large numbers stored digit-by-digit inside linked lists (most significant digit first) rather than strings, for tamper-evidence reasons. Given two such linked lists representing non-negative integers, compute their sum and return it in the same most-significant-digit-first linked-list form, without reversing the input lists.

Input: Two linked lists representing non-negative integers, most-significant-digit first.

Output: A linked list representing the sum, most-significant-digit first.

Example 1: *Input: l1 = [7,2,4,3], l2 = [5,6,4]*
Output: [7,8,0,7]

Example 2: *Input: l1 = [2,4,3], l2 = [5,6,4]*
Output: [8,0,7]

Describe the approach you used. Evaluate why the linked-list digit-addition variant (most-significant-digit-first) is harder to solve directly than the string-based multiplication problem, and what technique resolves this without reversing the lists.

Key Skills to be Addressed

Linked-list pointer manipulation, circular/modular indexing, digit-by-digit arithmetic simulation.

Applications

Big-integer arithmetic in cryptography, low-level data-structure merging in operating systems, bomb-disposal/cipher-style puzzle simulations, ledger and financial record systems.

Learning Outcome

Students will be able to manipulate linked-list pointers correctly during merges, apply modular indexing for circular data, and implement digit-by-digit arithmetic without relying on built-in big-integer types.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	2	Add Two Numbers	Linked-list digit arithmetic
2	66	Plus One	Digit-array simulation
3	415	Add Strings	String-based digit arithmetic

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including empty lists, a zero key in the circular cipher, and ledger strings containing a zero digit.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 2: Bit Manipulation Techniques

Bit Manipulation Techniques | Duration: 2 Hrs

CO Mapping	CO1 – Apply mathematical and number-theoretic foundations to formulate efficient solutions for computational problems.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	2 Hours
Total Hours of Problem Definition Implementation	1.5 Hours
Total Hours of Testing / Evaluation	0.5 Hours

Lab Aim

To apply XOR-based bit manipulation to isolate unique elements and maximize pairwise bitwise relationships in an array.

Problem 1

The Missing Roll Number

A teacher's assistant at a school where every student in a class is assigned a unique roll number from 0 to n collects the roll numbers of every student who submitted their exam paper. Exactly one student is absent, and their roll number is the only one missing from the collected list. Determine which single roll number never showed up, without maintaining a separate attendance sheet and without re-sorting the stack of papers.

Input: Integer array nums containing n distinct numbers, each in the range [0, n].

Output: The single integer in [0, n] that does not appear in nums.

Example 1: *Input: nums = [3,0,1]*
Output: 2

Example 2: *Input: nums = [0,1]*
Output: 2

Example 3: *Input: nums = [9,6,4,2,3,5,7,0,1]*
Output: 8

Problem 2

The Warehouse's Missing Partners

A warehouse's inventory log lists every product code twice — once per unit shipped in its safety-stock pair — except for exactly two product codes that arrived as singles. Using only O(1) extra memory and a single pass, identify both singleton codes.

Input: Integer array nums where every element appears exactly twice except two elements that appear exactly once.

Output: The two elements that appear only once, in any order.

Example 1: *Input: nums = [1,2,1,3,2,5]*
Output: [3,5]

Example 2: *Input: nums = [-1,0]*
Output: [-1,0]

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze why XOR-ing every element in an array against the full expected range isolates the single missing value in the Missing Roll Number problem.
2. **[Problem 2]** ***(Analyze)*** In the XOR-partition approach for the two-singleton problem, explain why isolating the lowest set bit of the cumulative XOR is guaranteed to separate the two unique numbers into different groups.
3. **[Problem 2]** ***(Evaluate)*** Evaluate why bitwise approaches to these problems use O(1) extra space compared to hash-set-based approaches, and what trade-off (if any) is made in return.

Supplementary Problem

The Encrypted Duel

A security analyst has intercepted a list of encryption keys and suspects that the strength of a potential attack depends on the largest possible XOR value obtainable by pairing any two keys from the list. Given the list of keys, determine the maximum XOR value achievable from any pair, in better than quadratic time.

Input: Integer array nums.

Output: The maximum XOR value obtainable from any pair of elements.

Example 1: *Input: nums = [3,10,5,25,2,8]*
Output: 28
Explanation: 5 XOR 25 = 28.

Example 2: *Input: nums = [0]*
Output: 0

Describe the approach you used. Design a Trie-based approach to find the maximum XOR of any two numbers in an array in better than $O(n^2)$ time, and explain how the Trie is traversed to greedily maximize each bit.

Key Skills to be Addressed

XOR-based isolation techniques, Trie-based greedy bit maximization, constant-space array scanning.

Applications

Error-detection and checksum systems, cryptographic key analysis, network packet analysis, low-memory embedded systems programming.

Learning Outcome

Students will be able to apply XOR properties to isolate unique elements in linear time and constant space, and extend bitwise reasoning to Trie-based maximum-XOR search.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	191	Number of 1 Bits	Bit counting
2	137	Single Number II	Bit-frequency isolation
3	231	Power of Two	Bitmask-based divisibility check

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including a single-element array and a duel key list containing only zeros.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 3: Number Theory Foundations

Number Theory Foundations | Duration: 2 Hrs

CO Mapping	CO1 – Apply mathematical and number-theoretic foundations to formulate efficient solutions for computational problems.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	2 Hours
Total Hours of Problem Definition Implementation	1.5 Hours
Total Hours of Testing / Evaluation	0.5 Hours

Lab Aim

To apply number-theoretic reasoning — factor counting, the extended Euclidean algorithm, and fast exponentiation — to solve constraint-based numeric problems.

Problem 1

The Records Office Stack Height

A records office stores every year's documents (numbered 1 through n) stacked into a single tower, where the total stack height is the product of every document count from 1 to n. Building safety compliance requires knowing how many reinforcement beams — one per trailing zero in the total stack-height number — are needed at that magnitude of weight. Given the number of documents, determine how many trailing zeroes the stack height contains.

Input: An integer n.

Output: An integer representing the number of trailing zeroes in n!.

Example 1: *Input: n = 3*
Output: 0
Explanation: 3! = 6, which has no trailing zero.

Example 2: *Input: n = 5*
Output: 1
Explanation: 5! = 120, which has one trailing zero.

Example 3: *Input: n = 25*
Output: 6

Problem 2

The Vault Override Codes

A regional bank uses a shared master-key policy across all its vault locks. For any two lock codes a and b issued to two branches, the head office needs an emergency override key derived directly from a and b, without either branch disclosing its raw code. Given any two lock codes a and b, determine their greatest common divisor g, together with integer coefficients x and y satisfying $a \cdot x + b \cdot y = g$, so that the override key can always be constructed on demand.

Input: Two integers a, b.

Output: Three integers g, x, y such that $g = \gcd(a, b)$ and $a \cdot x + b \cdot y = g$.

Example 1: *Input: a = 35, b = 15*
Output: g = 5, x = 1, y = -2
Explanation: 35×1 + 15×(-2) = 5.

Example 2: *Input: a = 240, b = 46*
Output: g = 2, x = -9, y = 47

Problem 3

The Session Key Generator

A university's login portal generates a one-time session key as base^exponent mod M, where exponent can be as large as 10^9. A naive loop multiplying base by itself exponent times is far too slow for the portal's response-time SLA, which requires the result within O(log exponent) multiplications. Compute base^exponent (and its version modulo M) within this bound.

Input: Base a, exponent n, and an optional modulus m.

Output: The value of a^n, and a^n mod m if m is given.

Example 1: *Input: a = 2, n = 10*
Output: 1024

Example 2: *Input: a = 3, n = 5, m = 100*
Output: a^n = 243, a^n mod m = 43

Example 3: *Input: a = 7, n = 0*
Output: 1

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze why the count of trailing zeroes in n! depends only on the number of factors of 5 in the product, not factors of 2, even though a trailing zero requires both a factor of 2 and a factor of 5.

2. **[Problem 2]** ***(Analyze)*** Analyze how the extended Euclidean algorithm's back-substitution produces valid Bézout coefficients, tracing the process for a=35, b=15.

3. **[Problem 3]** ***(Evaluate)*** Compare the iterative and recursive implementations of fast exponentiation in terms of stack depth, and justify which is safer for exponents up to 10^9.

Supplementary Problem

Splitting the Prize Pool Fairly

A tournament organizer must award prize shares based on the number of ways to choose winning teams from a large pool, but the count must be reported modulo a large prime p (to fit the platform's display limits), for pools far too large to compute directly via factorial division. Given the pool size n, the group size r, and the prime modulus p, determine the number of ways to choose r teams from n, modulo p.

Input: Integers n, r, and a prime modulus p.

Output: C(n, r) mod p.

Example 1: *Input: n = 5, r = 2, p = 1000000007*
Output: 10

Example 2: *Input: n = 10, r = 3, p = 13*
Output: 3
Explanation: C(10,3) = 120, and 120 mod 13 = 3.

Describe the approach you used. Using Fermat's Little Theorem and fast exponentiation, design a method to compute the modular inverse of a under a prime modulus p, and explain why p must be prime.

Key Skills to be Addressed

Number-theoretic factor counting, modular arithmetic, recursive back-substitution, fast exponentiation.

Applications

Cryptographic key generation, safety/engineering load calculations, combinatorial prize-distribution systems, checksum and hashing systems.

Learning Outcome

Students will be able to count prime factors efficiently, derive Bézout coefficients via the extended Euclidean algorithm, and compute large modular exponentials and combinations efficiently.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	204	Count Primes	Sieve-based number theory
2	326	Power of Three	Number-theoretic divisibility check
3	264	Ugly Number II	Number-theoretic sequence generation

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including n = 0 for the trailing-zero count, a and b where one is zero, and an exponent of zero.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%

Criteria	Weight (%)
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 4: Sliding Window & Greedy Optimization

Sliding Window & Greedy Optimization | Duration: 2 Hrs

CO Mapping	CO4 – Construct greedy algorithms and justify their correctness for optimization problems exhibiting the greedy-choice property.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	2 Hours
Total Hours of Problem Definition Implementation	1.5 Hours
Total Hours of Testing / Evaluation	0.5 Hours

Lab Aim

To apply dynamic-window and greedy feasibility reasoning to threshold-based and circular-resource problems.

Problem 1

The Warehouse's Minimum Truckload Window

A logistics coordinator has a target minimum load weight that a truck must carry, and a queue of package weights arriving in a fixed sequence. To avoid part-loading or re-routing, she wants the shortest contiguous run of packages whose combined weight meets or exceeds the target. Determine the length of the shortest such run, or report that no such run exists.

Input: Array nums of positive integers, integer target.

Output: Minimal length of a subarray with sum ≥ target, or 0 if none exists.

Example 1: *Input: target = 7, nums = [2,3,1,2,4,3]*
Output: 2

Example 2: *Input: target = 4, nums = [1,4,4]*
Output: 1

Problem 2

Reordering the Ceremonial Scroll

A scribe must rearrange a ceremonial scroll's text so that every consonant stays fixed in its original position, while the vowels among them are rearranged into non-decreasing order of their character values, without disturbing any consonant's position. Determine the resulting rearranged text.

Input: String s.

Output: String t with all consonants fixed in place and vowels sorted in non-decreasing ASCII order.

Example 1: *Input: s = "lEetcOde"*
Output: "lEOtcede"

Example 2: *Input: s = "lYmpH"*
Output: "lYmpH"
Explanation: No vowels present, so nothing changes.

Problem 3

The Round-Trip Fuel Puzzle

A delivery driver plans a circular route through n fuel stations, each offering a certain amount of fuel and requiring a certain amount of fuel to reach the next station. Determine the starting station index from which the driver can complete the entire circuit without running out of fuel, or report that no such start exists.

Input: Arrays gas and cost, each of length n.

Output: Starting station index for a feasible circular tour, or -1 if none exists.

Example 1: *Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]*
Output: 3

Example 2: *Input: gas = [2,3,4], cost = [3,4,3]*
Output: -1

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze why shrinking the window from the left the moment the sum meets the target (rather than resetting the window entirely) guarantees the shortest valid window is found.

2. **[Problem 2]** ***(Evaluate)*** Evaluate why extracting, sorting, and re-inserting only the vowels of a string (rather than sorting the entire string) is sufficient to solve the ceremonial-scroll reordering problem correctly.

3. **[Problem 3]** ***(Create)*** Design a proof sketch showing that if a feasible starting station exists in the Round-Trip Fuel Puzzle, the greedy approach (resetting the candidate start whenever the running balance goes negative) is guaranteed to find it.

Supplementary Problem

The Delivery Fleet's Peak Load Window

A fleet dispatcher tracks a continuous stream of delivery-load readings and must report, for every fixed-size window of consecutive readings as it slides across the entire stream, the maximum load value within that window at each position.

Input: Array nums, integer k (window size).

Output: Array of the maximum value in every window of size k as it slides across nums.

Example 1: *Input: nums = [1,3,-1,-3,5,3,6,7], k = 3*
Output: [3,3,5,5,6,7]

Example 2: *Input: nums = [1], k = 1*
Output: [1]

Describe the approach you used. Analyze how a monotonic deque maintains the maximum of a sliding window in amortized O(1) per step, rather than rescanning the window each time it slides.

Key Skills to be Addressed

Dynamic window maintenance, greedy feasibility reasoning, deque-based window-maximum tracking.

Applications

Network flow-control and rate-limiting, text-processing and formatting systems, delivery/logistics route planning, real-time monitoring dashboards.

Learning Outcome

Students will be able to maintain a dynamic window over an array to solve threshold-based problems, apply greedy reasoning to circular feasibility problems, and use a deque to track sliding-window extremes efficiently.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	3	Longest Substring Without Repeating Characters	Sliding window
2	763	Partition Labels	Greedy interval partitioning
3	455	Assign Cookies	Greedy matching

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including a target sum that is never reachable, a scroll with no vowels, and a fuel circuit with no feasible start.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 5: Sequence Alignment & Interval Greedy

Sequence Alignment & Interval Greedy | Duration: 2 Hrs

CO Mapping	CO3 – Design dynamic programming solutions by formulating optimal state-transition models for multi-stage optimization problems.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	2 Hours
Total Hours of Problem Definition Implementation	1.5 Hours
Total Hours of Testing / Evaluation	0.5 Hours

Lab Aim

To compare two sequences for shared structure and to make greedy interval/capacity-based scheduling decisions.

Problem 1

Aligning Two Manuscripts

A digital humanities team has two versions of an ancient manuscript, each transcribed independently, and wants to find the longest sequence of characters that appears in both transcriptions in the same relative order (not necessarily contiguous), to identify the "core" shared text.

Input: Strings text1, text2.

Output: Length of the longest common subsequence.

Example 1: *Input: text1 = "abcde", text2 = "ace"*
Output: 3

Example 2: *Input: text1 = "abc", text2 = "def"*
Output: 0

Problem 2

Loading the Lifeboats

A ship's crew must evacuate all passengers using the fewest possible lifeboats, where each boat carries at most two people and has a strict total weight limit. Determine the minimum number of boats needed.

Input: Array people (weights), integer limit.

Output: Minimum number of boats required.

Example 1: *Input: people = [1,2], limit = 3*
Output: 1

Example 2: *Input: people = [3,2,2,1], limit = 3*
Output: 3

Example 3: *Input: people = [3,5,3,4], limit = 5*
Output: 4

Problem 3

Scheduling the Conference Talks

A single conference room must host as many non-overlapping talks as possible from a list of proposed talks, each with a start and end time. Select the maximum number of talks that can be scheduled without any time overlap.

Input: Array of [start, end] talk intervals.

Output: Maximum number of non-overlapping talks that can be scheduled.

Example 1: *Input: intervals = [[1,3],[2,4],[3,5],[6,8]]*
Output: 3

Example 2: *Input: intervals = [[1,2],[2,3],[3,4]]*
Output: 3

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze how the LCS DP table's recurrence handles a character match versus a mismatch differently, and why this asymmetry is necessary for correctness.
2. **[Problem 2]** ***(Evaluate)*** Evaluate the greedy two-pointer strategy for the Lifeboats problem: why is it always optimal to pair the heaviest remaining person with the lightest remaining person?
3. **[Problem 3]** ***(Analyze)*** Analyze why sorting talks by end time (rather than start time) guarantees an optimal solution for the Conference Talks scheduling problem, using a counterexample where sorting by start time fails.

Supplementary Problem

The Cipher Correction Cost

A cryptographer needs the minimum number of single-character insert/delete/replace operations to transform one intercepted cipher string into a known reference string, to estimate how many transmission errors occurred.

Input: Strings word1, word2.

Output: Minimum number of insert/delete/replace operations to convert word1 to word2.

Example 1: *Input: word1 = "horse", word2 = "ros"*
Output: 3

Example 2: *Input: word1 = "intention", word2 = "execution"*
Output: 5

Describe the approach you used. Design a method to reconstruct the actual sequence of edits (not just the count) needed to transform one string into another, by backtracking through the completed DP table.

Key Skills to be Addressed

2D DP table construction for sequence alignment, greedy two-pointer pairing, interval-based greedy scheduling.

Applications

DNA/protein sequence alignment, version-control diff tools, evacuation and resource-pairing logistics, conference/exam scheduling systems.

Learning Outcome

Students will be able to formulate DP recurrences for sequence-alignment problems and apply greedy reasoning to capacity-constrained pairing and interval-scheduling problems.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	435	Non-overlapping Intervals	Interval-based greedy
2	646	Maximum Length of Pair Chain	Greedy interval chaining
3	1029	Two City Scheduling	Greedy sorting by cost difference

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including manuscripts with no common subsequence, a single passenger, and fully overlapping talks.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 6: Backtracking & Combinatorial Search

Backtracking & Combinatorial Search | Duration: 4 Hrs

CO Mapping	CO5 – Develop optimized solutions using advanced data structures and algorithmic paradigms to solve high-complexity problems within time and space constraints.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	4 Hours
Total Hours of Problem Definition Implementation	3 Hours
Total Hours of Testing / Evaluation	1 Hour

Lab Aim

To design and implement backtracking-based exhaustive search for enumeration and constraint-satisfaction problems.

Problem 1

The Pizza Shop Menu Catalog

An artisanal pizza shop wants a complete pre-priced catalog of every topping combination a customer could order, given a fixed list of unique toppings, so the kitchen can pre-price each one before printing the menu board. Generate every possible topping combination, including the empty order and the all-toppings order, with no combination listed twice.

Input: Array nums of unique integers.

Output: All possible subsets of nums (the power set), with no duplicate subsets.

Example 1: *Input: nums = [1,2,3]*
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Example 2: *Input: nums = [0]*
Output: [[],[0]]

Problem 2

The Relay Team Lineup

A coach of a relay team has a fixed roster of runners, each identified by a unique jersey number, and needs every possible running order the team could use, since the committee wants to review all valid lineups before the final one is locked in. Every lineup must use all the runners, each exactly once, in a different sequence. Produce the complete list of every possible running order.

Input: Array nums of distinct integers.

Output: All possible permutations of nums, in any order.

Example 1: *Input: nums = [1,2,3]*
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Example 2: *Input: nums = [0,1]*
Output: [[0,1],[1,0]]

Problem 3

The Vault Passphrase Grid

A vault's passphrase is hidden inside a grid of characters, spelled out by moving between orthogonally adjacent cells, without reusing any single cell twice within one attempt. Given the grid and the target passphrase, determine whether the passphrase can be traced out.

Input: An m x n grid of characters board, and a string word.

Output: Boolean — true if word exists in the grid.

Example 1: *Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"*
Output: true

Example 2: *Input: board = [["A","B"],["C","D"]], word = "ABCD"*
Output: false

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** For a topping list of size n, prove that the total number of subsets generated is 2^n, and explain how the backtracking recursion tree reflects this.

2. **[Problem 2]** ***(Evaluate)*** Compare generating permutations via backtracking-with-swapping against backtracking-with-a-used-array; evaluate the memory and readability trade-offs of each.

3. **[Problem 3]** ***(Analyze)*** Analyze why marking a grid cell as visited before recursing and un-marking it after returning is essential to the correctness of the passphrase search, using an example where omitting the un-mark step produces a wrong answer.

Supplementary Problem

Placing the Non-Conflicting Sentries

A fortress commander must place n sentries on an n×n grid of watchtowers such that no two sentries can see each other along any row, column, or diagonal. Determine every distinct arrangement of sentry placements that satisfies this constraint.

Input: Integer n.

Output: All distinct board configurations placing n sentries with no two attacking each other.

Example 1: *Input: n = 4*
Output: 2 distinct solutions
Explanation: ["·Q··","···Q","Q···","··Q·"] and ["··Q·","Q···","···Q","·Q··"].

Example 2: *Input: n = 1*
Output: 1 solution ("Q")

Describe the approach you used. Design the pruning conditions needed so invalid placements are rejected as early as possible in the recursion, rather than only checked at the end.

Key Skills to be Addressed

Backtracking recursion design, state restoration (mark/un-mark), constraint-based pruning.

Applications

Puzzle and pathfinding engines, configuration/test-case enumeration, constraint-satisfaction problems in scheduling and design, combinatorial security key-space analysis.

Learning Outcome

Students will be able to design correct backtracking recursions with proper state restoration and apply early-pruning strategies to constraint-satisfaction search problems.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

4 Hours (3 hrs implementation, 1 hr testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	77	Combinations	Backtracking enumeration
2	39	Combination Sum	Backtracking with repetition
3	212	Word Search II	Trie-accelerated grid backtracking

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including an empty topping list, a single-runner lineup, and a passphrase not present anywhere in the grid.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 7: Dynamic Programming on Sequences

Dynamic Programming on Sequences | Duration: 4 Hrs

CO Mapping	CO3 – Design dynamic programming solutions by formulating optimal state-transition models for multi-stage optimization problems.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	4 Hours
Total Hours of Problem Definition Implementation	3 Hours
Total Hours of Testing / Evaluation	1 Hour

Lab Aim

To formulate DP recurrences for probability-propagation, non-adjacent selection, and contiguous-subarray optimization problems.

Problem 1

The Champagne Pyramid

Glasses are stacked in a triangular pyramid, one glass in the first row, two in the second, and so on, with each glass in a row able to hold exactly one cup of champagne. Champagne is poured into the single topmost glass; once a glass is full, any excess overflow splits equally to the two glasses immediately below it, continuing down the pyramid, with any overflow from the bottom row simply spilling onto the floor. Given the number of cups poured, determine how full a specific glass in a specific row ends up being.

Input: poured (cups of champagne), query_row, query_glass (both 0-indexed).

Output: A fraction between 0 and 1 indicating how full the specified glass is.

Example 1: *Input: poured = 1, query_row = 1, query_glass = 1*
Output: 0.00000

Example 2: *Input: poured = 2, query_row = 1, query_glass = 1*
Output: 0.50000

Problem 2

The Night Robber's Dilemma

A professional robber is planning a street of houses, each holding a certain amount of money, but cannot rob two adjacent houses on the same night (a shared security link would alert the police). Determine the maximum amount that can be robbed.

Input: Array nums.

Output: Maximum amount that can be robbed.

Example 1: *Input: nums = [1,2,3,1]*
Output: 4

Example 2: *Input: nums = [2,7,9,3,1]*
Output: 12

Problem 3

The Trader's Best Streak

A data analyst reviewing a trader's daily profit/loss log wants to find the best possible continuous streak of days that maximizes total profit, out of all possible contiguous stretches of days.

Input: Array nums.

Output: Maximum sum of any contiguous subarray.

Example 1: *Input: nums = [-2,1,-3,4,-1,2,1,-5,4]*
Output: 6

Example 2: *Input: nums = [1]*
Output: 1

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze how excess champagne propagates from a full glass to the two glasses below it, and derive the recurrence relating a glass's fill level to the fill levels of the glasses above it.
2. **[Problem 2]** ***(Analyze)*** For the House Robber problem, derive the recurrence dp[i] = max(dp[i-1], dp[i-2] + nums[i]) and analyze why considering only the previous two states is sufficient.
3. **[Problem 3]** ***(Evaluate)*** Compare Kadane's algorithm with a divide-and-conquer approach to the maximum-subarray problem in terms of time complexity and implementation complexity.

Supplementary Problem

The Archivist's Longest Growing Trend

An archivist reviewing a century of annual rainfall records wants to find the longest stretch of years (not necessarily consecutive) in which rainfall values strictly increased, to identify the longest sustained wet trend in the region's history.

Input: Array nums.

Output: Length of the longest strictly increasing subsequence.

Example 1: *Input: nums = [10,9,2,5,3,7,101,18]*
Output: 4

Example 2: *Input: nums = [0,1,0,3,2,3]*
Output: 4

Describe the approach you used. Design the O(n log n) patience-sorting approach, and explain how it improves on the $O(n^2)$ DP recurrence.

Key Skills to be Addressed

Probability/quantity-propagation DP, non-adjacent-selection DP, contiguous-subarray optimization, patience-sorting optimization.

Applications

Resource allocation and budgeting, financial trend analysis, physical/liquid distribution simulations, climate and environmental trend analysis.

Learning Outcome

Students will be able to formulate DP recurrences for propagation-based, selection-based, and subsequence-growth problems, and apply optimized techniques where a faster alternative exists.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

4 Hours (3 hrs implementation, 1 hr testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	70	Climbing Stairs	Basic 1D DP recurrence
2	213	House Robber II	Circular-array DP variant
3	673	Number of Longest Increasing Subsequence	LIS with counting

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including a single house, an all-negative profit log, and a poured amount of zero cups.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 8: Monotonic Stack & Two-Pointer Patterns

Monotonic Stack & Two-Pointer Patterns | Duration: 4 Hrs

CO Mapping	CO5 – Develop optimized solutions using advanced data structures and algorithmic paradigms to solve high-complexity problems within time and space constraints.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	4 Hours
Total Hours of Problem Definition Implementation	3 Hours
Total Hours of Testing / Evaluation	1 Hour

Lab Aim

To apply monotonic-stack and two-pointer reasoning to reduce sequence-scanning problems from quadratic to linear time.

Problem 1

The Trader's Warmer-Day Countdown

A commodities trader logs the daily price of a stock and wants, for each day, to know how many days must pass before a strictly higher price occurs, to plan short-term hedges. The brute-force day-by-day comparison is too slow for the trader's tick-by-tick feed; find an approach that scales to a full year of minute-level data.

Input: Array temperatures.

Output: Array answer where answer[i] is the number of days to wait for a warmer temperature (0 if none).

Example 1: *Input: temperatures = [73,74,75,71,69,72,76,73]*
Output: [1,1,4,2,1,1,0,0]

Example 2: *Input: temperatures = [30,40,50,60]*
Output: [1,1,1,0]

Problem 2

The Skyline Water Catchment

A city planner has the heights of a row of buildings along a street (one unit wide each) and wants to know how much rainwater would be trapped between them after a storm, based solely on the height profile. Determine the total volume of water that would be caught.

Input: Array height of non-negative integers (width of each bar = 1).

Output: Total units of rainwater trapped.

Example 1: *Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]*
Output: 6

Example 2: *Input: height = [4,2,0,3,2,5]*
Output: 9

Problem 3

The Asteroid Field Simulation

A space traffic controller monitors a row of asteroids, each moving at the same speed either left or right along a single line, where an asteroid's size is given and its direction is indicated by the sign of its value. When two asteroids traveling toward each other meet, the smaller one is destroyed; if they are equal in size, both are destroyed. Determine the final state of the asteroid field after all such collisions have resolved.

Input: Array asteroids of integers (sign = direction, magnitude = size).

Output: Array representing the state of the asteroids after all collisions.

Example 1: *Input: asteroids = [5,10,-5]*
Output: [5,10]

Example 2: *Input: asteroids = [8,-8]*
Output: []

Example 3: *Input: asteroids = [10,2,-5]*
Output: [10]

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze why a monotonic (decreasing) stack correctly finds, for each day, the next warmer day in a single pass, rather than requiring a nested comparison loop.

2. **[Problem 2]** ***(Evaluate)*** Compare a two-pointer/prefix-based approach to the skyline water-catchment problem against a brute-force per-building max-scan approach; evaluate the difference in time complexity.

3. **[Problem 3]** ***(Analyze)*** Analyze why, in the asteroid-field simulation, a stack (rather than direct array scanning) correctly resolves chained collisions where one surviving asteroid may go on to collide with another.

Supplementary Problem

The Skyline Building Zone

An urban planner has the heights of a row of adjacent building plots and wants to identify the largest rectangular billboard zone that can be constructed using only contiguous plots, where the zone's height is limited by the shortest plot within its span. Determine the maximum possible area of such a zone.

Input: Array heights.

Output: Area of the largest rectangle achievable using contiguous bars.

Example 1: *Input: heights = [2,1,5,6,2,3]*
Output: 10

Example 2: *Input: heights = [2,4]*
Output: 4

Describe the approach you used. Explain how the stack-based approach generalizes the daily-temperatures monotonic-stack idea to a 2D area computation.

Key Skills to be Addressed

Monotonic stack maintenance, two-pointer/prefix-bound tracking, simulation of sequential state changes.

Applications

Stock/price-trend analysis tools, terrain and water-retention modeling, space traffic and collision simulations, real-estate zoning and layout optimization.

Learning Outcome

Students will be able to apply monotonic stacks and two-pointer techniques to reduce $O(n^2)$ sequence-scanning problems to O(n), and simulate sequential collision/elimination processes correctly.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

4 Hours (3 hrs implementation, 1 hr testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	496	Next Greater Element I	Monotonic stack
2	901	Online Stock Span	Streaming monotonic stack
3	402	Remove K Digits	Monotonic stack on digits

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including a strictly decreasing temperature sequence, a flat skyline, and an asteroid field with no collisions.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 9: Graph Algorithms

Graph Algorithms | Duration: 4 Hrs

CO Mapping	CO2 – Analyze and solve graph-theoretic problems by selecting appropriate graph algorithms based on problem structure and constraints.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	4 Hours
Total Hours of Problem Definition Implementation	3 Hours
Total Hours of Testing / Evaluation	1 Hour

Lab Aim

To apply graph traversal and greedy edge-selection to identify connectivity, ordering constraints, and minimum-cost spanning structures.

Problem 1

Mapping the Provinces

A country's road ministry has direct-connection data between cities represented as an adjacency matrix. A province is any group of cities reachable from one another, directly or indirectly. Determine how many distinct provinces exist.

Input: n x n matrix isConnected.

Output: Number of provinces.

Example 1: *Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]*
Output: 2

Example 2: *Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]*
Output: 3

Problem 2

The Course Prerequisite Auditor

A university registrar must verify that a proposed set of course prerequisites doesn't create an impossible-to-satisfy cycle (e.g., Course A requires B, which requires A). Given the total number of courses and a list of prerequisite pairs, determine whether all courses can be completed.

Input: numCourses, and array prerequisites where prerequisites[i] = [ai, bi].

Output: Boolean — true if all courses can be finished.

Example 1: *Input: numCourses = 2, prerequisites = [[1,0]]*
Output: true

Example 2: *Input: numCourses = 2, prerequisites = [[1,0],[0,1]]*
Output: false

Problem 3

Wiring the Smart Campus

A university wants to lay minimum-length cable between newly installed sensor points across campus, where the cost of connecting any two points is their Euclidean distance. Every point must end up connected, directly or indirectly. Determine the minimum total cable cost required to connect all the points.

Input: Array of (x, y) coordinates.

Output: Minimum total cost required to connect all the points.

Example 1: *Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]*
Output: 20

Example 2: *Input: points = [[3,12],[-2,5],[-4,1]]*
Output: 18

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Compare using DFS vs Union-Find to count provinces; analyze the time complexity of each on a dense n×n adjacency matrix.

2. **[Problem 2]** ***(Evaluate)*** For the course-prerequisite auditor, evaluate why detecting a cycle via DFS "gray/visiting" node-coloring is necessary, and what would go wrong if you only tracked "visited" without distinguishing in-progress nodes.

3. **[Problem 3]** ***(Analyze)*** Analyze why both Prim's-style and Kruskal's-style approaches are guaranteed to produce a minimum spanning tree, referring to the cut property.

Supplementary Problem

The Signal Broadcast Delay

A network operations team models their server cluster as a directed weighted graph, where an edge (u, v, w) means server u can signal server v with a delay of w. Given a signal broadcast from a source server k, determine the minimum time for the signal to reach all n servers, or report that it's impossible.

Input: n servers, edge list times[i] = [u, v, w], source server k.

Output: Minimum time for the signal to reach all servers, or -1 if impossible.

Example 1: *Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2*
Output: 2

Example 2: *Input: times = [[1,2,1]], n = 2, k = 2*
Output: -1

Describe the approach you used. Explain the modification needed to the shortest-path relaxation process, and why it fails if any edge weight is negative.

Key Skills to be Addressed

Graph traversal (DFS/BFS), Union-Find, topological ordering, greedy minimum-spanning-tree construction, shortest-path relaxation.

Applications

Network routing and infrastructure design, build-system/task dependency resolution, social-network community detection, GPS and navigation systems.

Learning Outcome

Students will be able to model relationships as graphs and apply traversal, ordering, and greedy edge-selection algorithms to solve connectivity, dependency, and minimum-cost network problems.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; graph visualization via networkx/matplotlib (optional); Git & GitHub for submission.

Total Hours of Engagement

4 Hours (3 hrs implementation, 1 hr testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	200	Number of Islands	Connected components via DFS/BFS on grid
2	210	Course Schedule II	Topological sort with output ordering
3	1135	Connecting Cities With Minimum Cost	Minimum spanning tree

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including a fully disconnected graph, a self-referential prerequisite pair, and a broadcast source that cannot reach every server.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%

Lab 10: Advanced String Dynamic Programming

Advanced String Dynamic Programming | Duration: 4 Hrs

CO Mapping	CO3 – Design dynamic programming solutions by formulating optimal state-transition models for multi-stage optimization problems.
Bloom's Taxonomy Level	L4 – Analyze / L5 – Evaluate / L6 – Create
Lab Duration	4 Hours
Total Hours of Problem Definition Implementation	3 Hours
Total Hours of Testing / Evaluation	1 Hour

Lab Aim

To formulate 2D DP recurrences for pattern-matching and palindrome-structure problems over strings.

Problem 1

Matching the Redacted Directive

An intelligence analyst has a redacted directive pattern where '?' stands for exactly one unknown character and '*' stands for any sequence of unknown characters (including none), and must determine whether a fully intercepted message matches the entire redacted pattern, not just part of it.

Input: String s, pattern p (containing '?' and '*').

Output: Boolean — true if p matches the entirety of s.

Example 1: *Input: s = "aa", p = "a"*
Output: false

Example 2: *Input: s = "aa", p = "*"*
Output: true

Example 3: *Input: s = "cb", p = "?a"*
Output: false

Problem 2

The Longest Mirrored Fragment

A linguist analyzing an ancient inscription wants to find the longest contiguous fragment of the inscription that reads identically forwards and backwards, to identify the inscription's ceremonial centerpiece.

Input: String s.

Output: The longest palindromic substring in s.

Example 1: *Input: s = "babad"*
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2: *Input: s = "cbbd"*
Output: "bb"

Problem 3

The Fortune Teller's Auspicious Segment

A translator working on an ancient digit scroll believes a segment of digits is "auspicious" if its digits could be rearranged into a palindrome. Find the longest such auspicious (contiguous) segment in the scroll, without checking every possible segment individually.

Input: String s consisting only of digits 0–9.

Output: Length of the longest substring whose digits could be rearranged into a palindrome.

Example 1: *Input: s = "3242415"*
Output: 5
Explanation: "24241" can be rearranged into the palindrome "24142".

Example 2: *Input: s = "12345678"*
Output: 1

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** ***(Analyze)*** Analyze how the wildcard-matching DP recurrence handles the '*' character differently from a literal character or '?', and why a 2D table is required.
2. **[Problem 2]** ***(Evaluate)*** Compare an $O(n^2)$ DP approach to the Longest Mirrored Fragment problem against an expand-around-center approach, in terms of time and space complexity.
3. **[Problem 3]** ***(Create)*** Design the bitmask-parity state used to solve the Fortune Teller's Auspicious Segment problem, explaining why at most one odd-count digit is allowed for a segment to be rearrangeable into a palindrome.

Supplementary Problem

Matching the Extended Directive

A follow-up intelligence directive uses an extended redaction notation where '.' stands for any single character and '*' means the preceding character may repeat zero or more times. Determine whether a fully intercepted message matches the entire extended pattern.

Input: String s, pattern p (containing '.' and '*').

Output: Boolean — true if p matches the entirety of s.

Example 1: *Input: s = "aa", p = "a"*
Output: false

Example 2: *Input: s = "aa", p = "a**"*
Output: true

Example 3: *Input: s = "ab", p = ".**"*
Output: true

Describe the approach you used. Evaluate how the extended directive's handling of "preceding-character-repeated-zero-or-more" differs from the simpler wildcard '*' in the main problem, and what additional DP transition this requires.

Key Skills to be Addressed

2D DP table construction for pattern matching, expand-around-center technique, bitmask-based parity state design.

Applications

Text search and pattern-matching engines, intelligence/redacted-message analysis, ancient-text and linguistic analysis, DNA motif and palindrome detection in bioinformatics.

Learning Outcome

Students will be able to formulate and implement 2D DP recurrences for wildcard and palindrome-structure string problems, and design bitmask-based state representations for parity-constrained substring problems.

Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design your own test data covering typical inputs, boundary/edge cases, and at least one stress/large-input scenario. State the expected output for each test case before running your code.

Tools / Technology to be Used

C / C++ / Java / Python 3, using VS Code or any terminal-based IDE; Git & GitHub for version-controlled submission.

Total Hours of Engagement

4 Hours (3 hrs implementation, 1 hr testing/modification/faculty evaluation)

Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems, along with your GitHub/journal documentation. The Supplementary Problem and LeetCode practice set below are for additional practice and are not required for submission.

Post Laboratory Work (LeetCode Practice)

Sr. No.	LC #	Problem	Concept
1	132	Palindrome Partitioning II	Palindrome-structure DP
2	115	Distinct Subsequences	2D string-matching DP
3	516	Longest Palindromic Subsequence	LCS-based palindrome DP

Rubrics (Lab Evaluation / Viva)

Criteria	Weight (%)
Program produces correct output for all given and edge-case inputs, including an empty pattern, a single-character string, and a scroll containing only one repeated digit.	40%
Student can explain their logic, choice of approach, and any optimizations applied.	25%
Student can describe at least one alternative strategy in plain English and identify its trade-offs.	20%
Code is well-structured, uses meaningful variable names, and includes basic comments.	15%